Story Splitting

When a story is too big to finish in an iteration, split it into thin, vertical slices that each deliver value.

Break Down Work · INVEST · SPIDR · Story Splitting Patterns · Vertical Slicing

RUNNING EXAMPLE

"As a shopper, I want to check out my basket so that I receive my order."

I — Independent
Can it be built and released on its own?

N — Negotiable
Is it a conversation, not a contract?

V — Valuable
Does a user or customer get value from it?

E — Estimable
Do we understand it well enough to size it?

S — Small
Can we finish it in a few days, well within an iteration?

T — Testable
Do we know how to prove it's done?

BEFORE YOU SPLIT: DOES THE STORY PASS INVEST? IF "SMALL" FAILS, SPLIT IT. IF "VALUABLE" FAILS, RETHINK IT.

SPLITTING PATTERNS: TRY THEM IN ROUGHLY THIS ORDER

1 Workflow Steps — PATHS

Split along the steps users take. Build the thinnest end-to-end path first, then add the extra steps.

FIRST	Basket → pay → confirmation
LATER	Gift wrap, delivery notes

2 Business Rules — RULES

Start with the simplest rule set, then add each rule variation as its own story.

FIRST	Standard delivery, no discounts
LATER	Discount codes, delivery limits

3 Happy / Unhappy Paths — PATHS

Deliver the success path first, then handle errors and edge cases in follow-up stories.

FIRST	Card payment succeeds
LATER	Declined card, timeout, retry

4 Data Variations — DATA

Support one type of data first, then widen to more types, formats or sources.

FIRST	UK addresses
LATER	International addresses

5 Interfaces — INTERFACES

Start with one channel, device or input method, then add the others.

FIRST	Web checkout, card only
LATER	Mobile app, Apple Pay, PayPal

6 Simple → Complex — SIMPLIFY

Ask "what's the simplest version that could work?", then defer the complicated parts.

FIRST	Flat shipping fee
LATER	Price by weight and zone

7 Operations (CRUD) — OPERATIONS

Split "manage X" into create, read, update and delete. Ship the ones users need first.

FIRST	Save and view a card
LATER	Edit and delete saved cards

8 Defer Performance — QUALITY

Make it work, then make it fast or scale. Agree the minimum bar in the Definition of Done.

FIRST	100 orders an hour
LATER	Black Friday peak load

9 Spike First — SPIKE

Too unknown to split? Timebox research first. The output is knowledge, not code.

FIRST	1-day spike: choose a provider
LATER	Split the build using 1–8

Still too big?

- Combine patterns: workflow first, then rules within one step
- Find the 80/20: which slice gives the most value for the least effort?
- Ask what could be dropped entirely, not just deferred

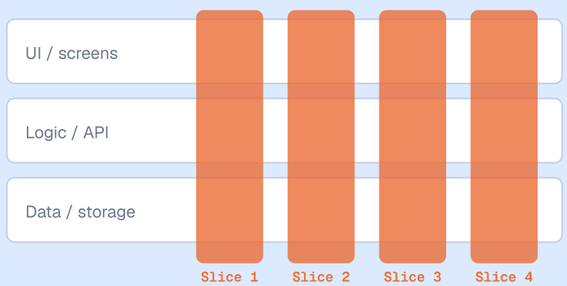
Slice Vertically, Not Horizontally

✗ HORIZONTAL

- UI / screens — Story 1
- Logic / API — Story 2
- Data / storage — Story 3

Nothing is usable until every layer is finished.

✓ VERTICAL



Each slice is usable and testable on its own.

Is It a Good Split?

AFTER SPLITTING, CHECK EACH SLICE

- Delivers value or learning by itself
- Fits comfortably in an iteration: aim for several per iteration
- Has clear acceptance criteria
- The team can estimate it
- Still passes INVEST

CHOOSE THE SPLIT THAT…

- Lets you deprioritise or drop a slice
- Makes slices of roughly equal size
- Gets user feedback soonest
- Tackles the biggest risk first
- Exposes low-value work you can skip

If no slice can be dropped, try a different pattern.

"What's the thinnest slice that still delivers value to a user?"

"If we could only ship half of this, which half?"

© 2026 Eric Parker · CC BY 4.0 · eparker71.github.io/better-delivery-toolkit

INVEST (Bill Wake) · SPIDR (Mike Cohn) · Splitting Patterns (Richard Lawrence)